

MOON: A Modality Conversion-Based Efficient Multivariate Time Series Anomaly Detection

Yuanyuan Yao , Yuhan Shi, Lu Chen , Ziquan Fang , *Member, IEEE*, Yunjun Gao , *Senior Member, IEEE*, Leong Hou U , *Member, IEEE*, Yushuai Li , *Senior Member, IEEE*, and Tianyi Li , *Member, IEEE*

Abstract—Multivariate time series (MTS) anomaly detection identifies abnormal patterns where each timestamp contains multiple variables. Existing MTS anomaly detection methods fall into three categories: reconstruction-based, prediction-based, and classifier-based methods. However, these methods face three key challenges: (1) Unsupervised learning methods, such as reconstruction-based and prediction-based methods, rely on error thresholds, which can lead to inaccuracies; (2) Semi-supervised methods mainly model normal data and often underuse anomaly labels, limiting detection of subtle anomalies; (3) Supervised learning methods, such as classifier-based approaches, often fail to capture local relationships, incur high computational costs, and are constrained by the scarcity of labeled data. To address these limitations, we propose MOON, a supervised modality conversion-based multivariate time series anomaly detection framework. MOON enhances the efficiency and accuracy of anomaly detection while providing detailed anomaly analysis reports. First, MOON introduces a novel multivariate Markov Transition Field (MV-MTF) technique to convert numeric time series data into image representations, capturing relationships across variables and timestamps. Since numeric data retains unique patterns that cannot be fully captured by image conversion alone, MOON employs a Multimodal-CNN to integrate numeric and image data through a feature fusion model with parameter sharing, enhancing training efficiency. Finally, a SHAP-based anomaly explainer identifies key variables contributing to anomalies, improving interpretability. Extensive experiments on six real-world MTS datasets demonstrate that MOON outperforms six state-of-the-art methods by up to 93% in efficiency, 4% in accuracy and, 10.8% in interpretation performance.

Index Terms—Anomaly detection, interpretable system, multivariate time series.

I. INTRODUCTION

MULTIVARIATE time series anomaly detection identifies unusual patterns or behaviors across multiple variables over time. It benefits a wide range of real-life applications,

including finance [38], healthcare [3], and industrial monitoring [14], [44], where timely detection of anomalies can lead to significant improvements in decision-making and framework reliability [15], [53]. In recent years, deep learning techniques have been widely applied to time series anomaly detection and achieved superior performance.

In multivariate time series anomaly detection, the scarcity of labeled anomalies and high annotation costs pose major challenges. To address these, unsupervised methods are widely used [5], [12], [13], [26], [39], [42], [49], modeling normal behavior and detecting deviations via prediction or reconstruction errors. However, unsupervised methods typically rely on large amounts of continuous and stable normal data as the basis for modeling. In real-world applications, such as in financial or industrial systems, data often fluctuates due to external shocks, making it difficult to obtain stable normal sequences. Moreover, the definition of “normal” is often ambiguous and prone to labeling errors, where certain anomalous points may be mistakenly marked as normal, further limiting the adaptability of these methods. Semi-supervised methods attempt to combine a small amount of anomalous labels with unlabeled data, improving performance while controlling label dependence. However, they still rely on large amounts of stable normal data, limiting effectiveness in dynamic environments, while scarce anomalies hinder learning complex patterns. In contrast, supervised methods, while exhibiting strong discriminative performance when sample labels are available [9], [50], [51], are highly dependent on labels. In scenarios with scarce samples, they are prone to overfitting, leading to insufficient generalization capability.

The interpretability of anomaly detection methods is also crucial for understanding the root causes of anomalies. Existing methods [8], [27], [29], [34], [39], [42], [43] often rely on differences between reconstructed and ground-truth data to identify variables causing anomalies. However, they treat all models as closed-box models, ignoring unique structures and offering *low interpretability*. Thus, we aim to develop an *accurate, efficient, and interpretable* anomaly detection framework.

Accurate anomaly detection: To achieve accurate and robust anomaly detection under limited supervision, it is crucial to effectively leverage scarce anomaly labels, enhancing the model’s ability to recognize and model abnormal patterns with minimal annotation cost. Multimodal data improves performance by combining complementary information from different sources, thereby enhancing the overall result quality. However, collecting

Received 21 January 2025; revised 6 September 2025; accepted 30 September 2025. Date of current version 25 November 2025. This work was supported by NSFC under Grant 62472377, Grant 62402431, Grant 62025206, and Grant U23A20296. Recommended for acceptance by D.-N. Yang. (Corresponding author: Lu Chen.)

Yuanyuan Yao, Yuhan Shi, Lu Chen, Ziquan Fang, and Yunjun Gao are with the College of Computer Science, Zhejiang University, Hangzhou 310027, China (e-mail: yoyoyao@zju.edu.cn; shiyuhan@zju.edu.cn; luchen@zju.edu.cn; zq-fang@zju.edu.cn; gaoyj@zju.edu.cn).

Leong Hou U is with the Department of Computer and Information Science, University of Macau, Macau (e-mail: ryanlhu@um.edu.mo).

Yushuai Li and Tianyi Li are with the Department of Computer Science, Aalborg University, 9220, Aalborg, Denmark (e-mail: yusli@cs.aau.dk; tianyi@cs.aau.dk).

Digital Object Identifier 10.1109/TKDE.2025.3622154

TABLE I
MODAL CONVERSION TIME (SECONDS) FOR MTS

Datasize	4000	6000	8000	10000
Extended MTF	5428.81	12176.4	21752.41	34862.44
MV-MTF	4.44	6.11	7.65	9.25

multimodal data is often costly and impractical, while numeric time series data is readily available.

To address this, we propose Multivariate Markov Transition Field (MV-MTF), which explicitly encodes variable-to-variable and time-to-time transitions in a 2D format to enhance anomaly representation by capturing structural dynamics that are difficult to extract from raw sequences. However, MV-MTF may lose fine-grained numerical details that are crucial for identifying point anomalies or small deviations. To compensate for this, we integrate MV-MTF with raw numerical features using a shared-parameter multimodal CNN. This architecture allows the model to jointly learn global structural patterns and local numerical variations [6], [17]. The multimodal-CNN employs convolutional kernels with varying receptive fields to capture multi-scale features from both the image-like representations and the raw time series. These kernels effectively extract information across variable and timestamp dimensions, enabling robust multimodal feature fusion. To further enhance the model's ability to process multimodal data, we incorporate a Multimodal Attention mechanism within Multimodal-CNN. This mechanism identifies and focuses on the most relevant features across different modalities, enabling the model to prioritize key information effectively. By integrating this attention mechanism, Multimodal-CNN achieves greater sensitivity and accuracy in detecting anomalous patterns.

Efficient anomaly detection: Efficient modal conversion directly contributes to efficient anomaly detection by reducing the computational overhead associated with processing large datasets. Markov Transition Field (MTF) is an effective modal conversion method that transforms time-series data into image representations for analysis. It captures local temporal information, representing short-term dependencies between consecutive or nearby data points [25], [46]. However, existing MTF methods primarily focus on univariate data [25], [46] and are computationally intensive, as they calculate transitions between every pair of data points. A straightforward extension of MTF to MTS is calculating transitions across different variables and timestamps. As shown in Table I, the extended MTF incurs very high processing times, with 10,000 data points requiring nearly 10 hours to process. Moreover, combining data from both modalities may increase the training cost.

To address these limitations, we propose an optimized MV-MTF strategy to improve efficiency of modal conversion, we simplify the time dimension. Since the influence between variables decreases with temporal distance, we consider only the impact of values from the previous time step on the current time. This reduces computational complexity from $O(n^2)$ to $O(n)$, significantly accelerating MV-MTF while preserving essential local information. As shown in Table I, with 10,000 data points, MV-MTF processes 10,000 data points in just 9.25 seconds, a

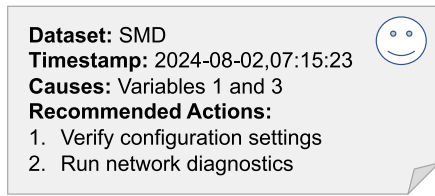


Fig. 1. An example of an anomaly report.

99.97% reduction compared to Extended MTF. Furthermore, in the multimodal-CNN module, we adopt a parameter-sharing strategy to reduce the overall number of trainable parameters, which not only decreases the memory footprint but also accelerates model convergence. This contributes to a more efficient training process and significantly shortens the overall training time.

Interpretable anomaly detection: Interpretability is one of the core objectives of our anomaly detection framework. For multi-variate time series detected as anomalous, the anomaly typically involves only a subset of variables. Accordingly, solving the anomalies requires not only determining whether an anomaly has occurred but also precisely localizing the affected variables. Existing threshold-based interpretability methods [39], [42] face two main limitations in anomaly detection. First, error score threshold-based methods risk misjudgment or omission. Second, they lack intuitive, model-specific explanations, limiting their interpretability.

To address these issues, we propose a high-interpretability method leveraging multimodal information. Gradient Shapley Additive Explanations (SHAP) [1] values are used to quantify the impact of other variables on a target variable and assess the current variable's contribution to anomaly occurrence. This enables a multi-dimensional evaluation of anomalies, clarifying how various factors contribute to the detection process.

Next, we apply a weighted ranking method to prioritize variables based on their contributions to anomaly detection, generating a ranked list of the most likely causes. To ensure reliability, we introduce an evaluation module that validates top-ranked variables, reducing false positives and enhancing the robustness of the explanations. Finally, we categorize anomalies into specific types based on expert insights. By clustering the data and constructing classifiers, we identify patterns that refine anomaly categorization.

A comprehensive anomaly report is then generated, as exemplified in Fig. 1. The report highlights key information about the anomaly, including (i) the dataset (SMD), (ii) the timestamp (2024-08-02, 07:15:23), (iii) the contributing variables (Variables 1 and 3), and (iv) actionable recommendations (verifying configuration settings and running network diagnostics). The report helps users quickly understand and address anomalies and enhances framework interpretability and reliability.

We integrate the above novel techniques to propose MOON, a **modality conversion-based anomaly detection framework** for MTS that is accurate, efficient, and interpretable. First, MOON employs the Multimodal-CNN to represent and fuse multimodal data while training a classifier for accurate anomaly detection.

Next, it efficiently converts numeric MTS data into image data using a new MV-MTF technique. Finally, MOON integrates an interpretability module to refine anomaly categories and generate detailed anomaly analysis reports.

To sum up, we have made the following key contributions:

- We present MOON, a modality conversion-based MTS anomaly detection framework, that efficiently and effectively supports MTS anomaly detection.
- We propose a supervised classification Multimodal-CNN, which integrates numerical MTS data and image representations using convolution kernels with varying receptive fields and a Multimodal Attention mechanism. It multi-scale features and prioritizes key information, enhancing anomaly detection accuracy.
- We propose a MV-MTF technique designed to simultaneously capture transitions across both time and variables. By leveraging the strong temporal dependencies in time-series data, we further simplify the calculation of transitions between variables, accelerating the modal conversion.
- We present a SHAP-based anomaly explainer that integrates multimodal data to compute SHAP values and incorporates a reliable evaluation module to enhance accuracy. By categorizing anomalies into distinct types, it significantly improves anomaly detection interpretability.
- We conduct experiments on six real-world datasets. The results demonstrate that MOON outperforms six state-of-the-art methods by up to 93% in efficiency, 4% in accuracy and, 10.8% in interpretation performance.

The rest of this paper is organized as follows. We provide the preliminaries in Section II. Section III provides our framework overview and main components of our framework. Section IV presents the experimental results. We review related work in Section V, and conclude the paper in Section VI.

II. PRELIMINARIES

A. Problem Definition

Definition 1 (Multivariate time series). A multivariate time series $X = (x_1, x_2, \dots, x_n)$ is a sequence of numerical observations ordered by time, where x_t ($1 \leq t \leq n$) is a c -dimensional vector ($c > 1$) representing the observation at timestamp t , and n is the length of the time series. The dimension of X is $n \times c$. For the v^{th} variable ($1 \leq v \leq c$), X^v represents its univariate time series, and x_t^v denotes its value at timestamp t .

Definition 2 (Anomaly detection). Given a training time series X , anomaly detection predicts $Y = \{y_t\}_{t=1}^{\hat{n}}$ for any unseen test time series $\hat{X}$ of length $\hat{n}$ with the same modality as X . $y_t \in \{0, 1\}$ indicates whether the data point at timestamp t of $\hat{X}$ is anomalous ($y_t = 1$ denotes anomalous points).

Definition 3 (Anomaly interpretability). Given an anomalous time series data point $\hat{x}_t$ at timestamp t with c variables, anomaly interpretability identifies the variables that contributed to classifying $\hat{x}_t$ as anomalous.

Example 1. Consider a multivariate time series $X = (x_1, x_2, x_3)$, where each x_t ($1 \leq t \leq 3$) has three variables: temperature, pressure, and humidity. The observations are $x_1 = [30, 101325, 20]$, $x_2 = [32, 101300, 21]$,

and $x_3 = [50, 101310, 20]$. Hence, $X^1 = [30, 32, 50]$, $X^2 = [101325, 101300, 101300]$, and $X^3 = [20, 21, 20]$. Anomaly detection identifies x_3 as an anomalous data point ($y_3 = 1$). Anomaly interpretability determines that the temperature value $x_3^1 = 50$ is the primary contributor to the anomaly, while pressure $x_3^2 = 101310$ and humidity $x_3^3 = 20$ remain normal.

B. Markov Transition Field Technology (MTF)

MTF techniques [25], [46] are primarily designed for converting univariate time series data to image data. For a univariate time series $X = (x_1, \dots, x_n)$ with a single variable ($c = 1$), data points are first discretized by mapping continuous values to discrete bins, reducing computational cost. Let Q denote the number of bins used to partition the data range. After discretization, each data point x_t is assigned to a bin with identifier q_i ($i \in [1, Q]$). A $Q \times Q$ state transition matrix W is then constructed, where W_{ij} represents the transition probability from bin q_i to bin q_j ($1 \leq i, j \leq Q$).

$$W = \begin{bmatrix} w_{11} & | & P_{11} & \cdots & w_{1Q} & | & P_{1Q} \\ w_{21} & | & P_{21} & \cdots & w_{2Q} & | & P_{2Q} \\ \vdots & & & & \vdots & & \\ w_{Q1} & | & P_{Q1} & \cdots & w_{QQ} & | & P_{QQ} \end{bmatrix}, \quad (1)$$

where w_{ij} denotes the transition probability in P_{ij} from q_j to q_i , with $P_{ij} = P(x_t \in q_i \mid x_{t-1} \in q_j)$, and t denotes any given time.

Example 2. Given three bins $q_1 = [0, 0.2)$, $q_2 = [0.2, 0.3)$, $q_3 = [0.3, 0.4)$, and a univariate time series $X = (0.1, 0.3, 0.05, 0.4, 0.15, 0.2)$, the data points are discretized as follows: x_1, x_3 , and x_5 are classified into q_1 , x_2 and x_4 into q_3 , and x_6 into q_2 . The resulting converted time series is $(1, 3, 1, 3, 1, 2)$. For the state transition matrix, $w_{13} = 2/3$, where 2 represents the two occurrences of the consecutive pair $(1, 3)$ in the converted series, and 3 is the total occurrences of pairs starting with 1 ($1 \leq i \leq 3$).

Based on the obtained state transition matrix W , an $n \times n$ Markov Transition Field (MTF) matrix M is constructed to capture the transition probabilities between states (i.e., timestamps), where m_{ij} represents the transition probability of state i to the state j , and its value equals to w_{ab} (i.e., $x_i \in q_a$, and $x_j \in q_b$). Note that, the obtained MTF matrix M is typically viewed as an image.

Example 3. Continuing Example 2, $m_{12} = w_{13}$ due to $x_1 \in q_1$ and $x_2 \in q_3$, $m_{23} = w_{31}$ due to $x_2 \in q_3$ and $x_3 \in q_1$, and $m_{34} = w_{13}$ due to $x_3 \in q_1$ and $x_4 \in q_3$.

C. Shapley Additive Explanations (SHAP)

SHAP [1] is a unified framework for interpreting the output of machine learning models, grounded in cooperative game theory. The theoretical foundation of SHAP is based on the Shapley value, which provides a fair distribution of payoffs to players depending on their contribution to the total payoff in a cooperative game. In machine learning, SHAP values assign an importance value to each feature, representing its contribution to the model's output. Given a model f and an instance x , the

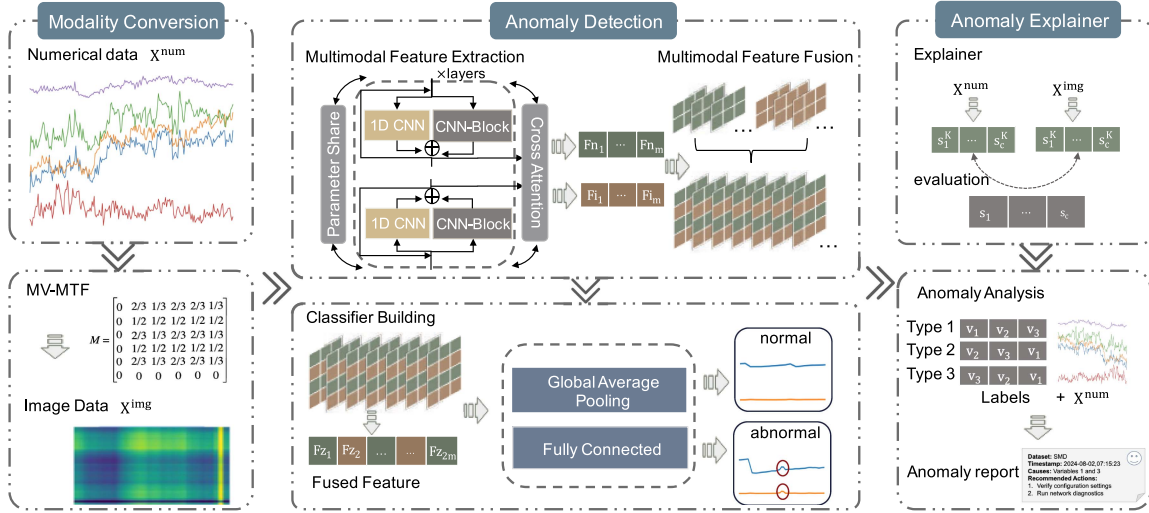


Fig. 2. Framework overview.

SHAP value $\phi_i(x)$ for feature i is computed as:

$$\phi_i(x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_x(S \cup \{i\}) - f_x(S)], \quad (2)$$

where F is the set of all features, S is a subset of F excluding feature i , $|S|$ is the number of features in subset S , and $f_x(S)$ is the model output for the subset S with the instance x .

The SHAP value computation ensures that the sum of SHAP values for all features equals the difference between the model output and the expected output. If two features contribute equally to all subsets, they receive equal SHAP values.

III. FRAMEWORK OVERVIEW

Fig. 2 illustrates an overview of MOON, comprising modal conversion, anomaly detection, and anomaly explainer.

- **Modal conversion.** To capture the local information between multivariate time series, we use the proposed MV-MTF technique to efficiently calculate the transition probabilities from other variables to a specific variable, thereby converting the multivariate time series into an image.
- **Anomaly detection.** To support two different data modalities (i.e., numerical time series data and image data), we propose Multimodal-CNN that utilizes a parameter sharing mechanism to cross-extract features from two modalities, significantly reducing computational complexity and parameter count. In addition, a multi-modal attention mechanism and a separable convolution-based feature fusion technique are introduced to combine the features from both modalities, thereby improving the classifier's performance.
- **Anomaly explainer.** To provide a user-friendly anomaly explainer, we generate an anomaly detection report for each detected anomaly. Specifically, for the anomalous time series data points, we use the kernel explainer and gradient explainer to generate SHAP values based on numerical and image time series data, respectively. Next, we use these

weighted SHAP values to identify the variables causing the anomalies. In addition, we train an explainer classifier to refine the anomaly categories.

The online anomaly detection process includes the following three steps. (i) MTS data is converted into image data using MV-MTF. (ii) Both the original MTS data and the converted image data are simultaneously fed into a Multimodal-CNN to determine if the data is anomalous. (iii) For data identified as anomalous, it is processed by an interpretable classifier to classify the anomaly type and generate a detailed anomaly detection report.

IV. FRAMEWORK DESIGN

A. Modality Conversion

To enable efficient multivariate time series conversion, we introduce MV-MTF technology, which generates an image capturing the time dependency relationships between variables, as shown in the left part of Fig. 2.

Let $X^u = \{x_1^u, x_2^u, \dots, x_n^u\}$ represent the time series for variable u ($u \in [1, c]$), mapped into Q_u distinct bins. Similarly, let $X^{u'} = \{x_1^{u'}, x_2^{u'}, \dots, x_n^{u'}\}$ represent the time series for another variable u' mapped into $Q_{u'}$ distinct bins. To determine an appropriate bin count Q_u for each variable u , we search over a candidate set of bin counts Q . For each $Q_u \in Q$, we apply quantile-based binning to discretize X^u into Q_u bins. The resulting discretized sequence Q^u is then evaluated using entropy:

$$H(Q^u) = - \sum_{i=0}^{|Q_u|-1} p_i \log p_i, \quad p_i = \frac{1}{n} \sum_{t=1}^n I(Q_t^u = i) \quad (3)$$

where $I(\cdot)$ is an indicator function that equals 1 if the t -th sample falls into the i -th bin, and 0 otherwise. We then select the optimal bin count that maximizes the entropy: $Q_u^* = \arg \max_{Q_u \in Q} H(Q^u)$.

With the optimal bin count determined for each variable, we proceed to analyze temporal transitions between discretized states, in contrast to hierarchical binning [28], which first partitions the data by layers or groups before discretization. For two consecutive timestamps, data points x_{t-1}^u and $x_t^{u'}$ from variables u and u' are classified into bins q_i^u ($i \in [1, Q_u]$) and $q_j^{u'}$ ($j \in [1, Q_{u'}]$), respectively. A $Q_u \times Q_{u'}$ state transition matrix W is then calculated as follows.

$$W = \left[w_{ij} \mid P \left(x_t^{u'} \in q_j^{u'} \mid x_{t-1}^u \in q_i^u \right) \right], \quad (4)$$

where $1 \leq i \leq Q_u$, $1 \leq j \leq Q_{u'}$, and $w_{i,j}$ is the transition probability $P(x_t^{u'} \in q_j^{u'} \mid x_{t-1}^u \in q_i^u)$ from q_i^u in variable X^u to $q_j^{u'}$ in variable $X^{u'}$.

Based on the obtained state transition matrix W , we compute the $n \times n$ Markov Transition Field matrix M . For any data point x_i^u of variable u at timestamp i and data point $x_j^{u'}$ of variable u' at timestamp j (x_i^u is classified into q_a^u , while $x_j^{u'}$ is classified to $q_b^{u'}$), the value m_{ij} in M equals to the state transition probability w_{ab} in W , which represents the probability of $x_t^{u'}$ at time t belonging to $q_b^{u'}$ given that the data in x_{t-1}^u at time $t-1$ belonging to q_a^u . This gives us the Markov Transition Field matrix $M_{u,u'}$ for variables u and u' .

Example 4. Given two time series for variables u and u' : $X^u = (0.1, 0.3, 0.05, 0.4, 0.15, 0.2)$ and $X^{u'} = (0.2, 0.3, 0.45, 0.3, 0.35, 0.4)$, with their respective bins defined as $Q^u : q_1^u = [0, 0.2), q_2^u = [0.2, 0.3), q_3^u = [0.3, 0.4)$ and $Q^{u'} : q_1^{u'} = [0.2, 0.3), q_2^{u'} = [0.3, 0.4), q_3^{u'} = [0.4, 0.5)$.

The sequence X^u is classified as follows: x_1^u, x_3^u , and x_5^u are classified into q_1^u ; x_2^u and x_4^u are classified into q_2^u ; while x_6^u is classified into q_3^u . For the sequence $X^{u'}$: $x_1^{u'}$ is classified into $q_1^{u'}$; $x_2^{u'}$, $x_4^{u'}$, and $x_5^{u'}$ are classified into $q_2^{u'}$; while $x_3^{u'}$ and $x_6^{u'}$ are classified into $q_3^{u'}$. Thus, we obtain the converted time series $X_q^u = (1, 3, 1, 3, 1, 2)$ and $X_q^{u'} = (1, 2, 3, 2, 2, 3)$ respectively. The transition probability $w_{12} = \frac{2}{3}$, where 2 indicates two occurrences of the pair (1, 2) in consecutive time steps from q_{t-1}^u to $q_t^{u'}$, and 3 represents the total occurrences of pairs starting from q_1^u ($1 \leq i \leq 3$). The corresponding matrices W and M are shown below.

$$W = \begin{bmatrix} 0 & 2/3 & 1/3 \\ 0 & 0 & 0 \\ 0 & 1/2 & 1/2 \end{bmatrix}$$

$$M = \begin{bmatrix} 0 & 2/3 & 1/3 & 2/3 & 2/3 & 1/3 \\ 0 & 1/2 & 1/2 & 1/2 & 1/2 & 1/2 \\ 0 & 2/3 & 1/3 & 2/3 & 2/3 & 1/3 \\ 0 & 1/2 & 1/2 & 1/2 & 1/2 & 1/2 \\ 0 & 2/3 & 1/3 & 2/3 & 2/3 & 1/3 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

According to W , the probability $\frac{2}{3}$ from q_1^u to $q_2^{u'}$ is assigned to m_{12} , since $x_1^u \in q_1^u$ and $x_2^{u'} \in q_2^{u'}$.

Note that, we have more than two variables in a multivariate time series data, thus, for a specific variable u , the Markov Transition Field matrices $M_{u,k}$ between variable u and all other

variables k ($1 \leq k \leq c$ and $k \neq u$) are computed. These matrices are then averaged to obtain a new Markov Random Field matrix $M_{u,\cdot}$, as shown in Equation (5).

$$M_{u,\cdot} = \sum_{k=1, k \neq u}^c \omega_{u,k} \times M_{u,k}, \quad (5)$$

where $\omega_{u,k}$ is the weight parameter for $M_{u,k}$, denoting the influence degree of k on u . The previously computed MTF matrix $M_{u,u}$ for variable u (2) is weighted and summed with $M_{u,\cdot}$ to obtain the Multivariate Markov Transition Field matrix M_u for variable u , as shown in Equation (6),

$$M_u = \alpha \times M_{u,u} + (1 - \alpha) \times M_{u,\cdot}, \quad (6)$$

where α is the weight parameter to control the influence degree of the same variable on the transition probability.

For a multivariate time series data X with c variables, we obtain the MV-MTF M_X for the multivariate time series data X by summing the MTF matrices of all variables, as shown in Equation (7).

$$M_X = \sum_{u=1}^c M_u, \quad (7)$$

where M_u is the MV-MTF matrices for all variables u ($u \in [1, c]$).

Time complexity analysis: The computational complexity of each MTF matrix computation is $O(n^2)$. In particular, the MTF matrix M requires calculating the values for all pairs in X^u and $X^{u'}$, resulting in very high complexity.

However, for time series data, the temporal relevance is crucial, and calculating the relationship between two timestamps time points do not make sense. Therefore, we optimize M as:

$$M_{ij} = \begin{cases} m_{i,j}, & \text{if } j = i + 1 \\ 0, & \text{otherwise} \end{cases}, \quad (8)$$

Each element in M represents the transition relationship between two consecutive timestamps. In this way, the complexity of Markov Transition Field matrix computation is reduced from $O(n^2)$ to $O(n)$.

B. Anomaly Detection

We present Multimodal-CNN with two key components: multimodal feature extraction and multimodal feature fusion.

1) Multimodal Feature Extraction: This component incorporates a unified framework for extracting features from both numerical and image data, as illustrated in Fig. 3. It begins by initializing parameters, with a focus on designing the receptive field for the CNN-Block. Using this design, the Multimodal-CNN extracts multi-scale features, capturing critical information through a multimodal attention mechanism. Moreover, parameter sharing is employed across modalities, further enhancing feature extraction performance. The detailed process is described below.

Receptive field design. In the multimodal feature extraction module, determining the receptive field size of the 1D-CNN is crucial, as it directly influences the range of features captured during convolution. To handle features at different scales

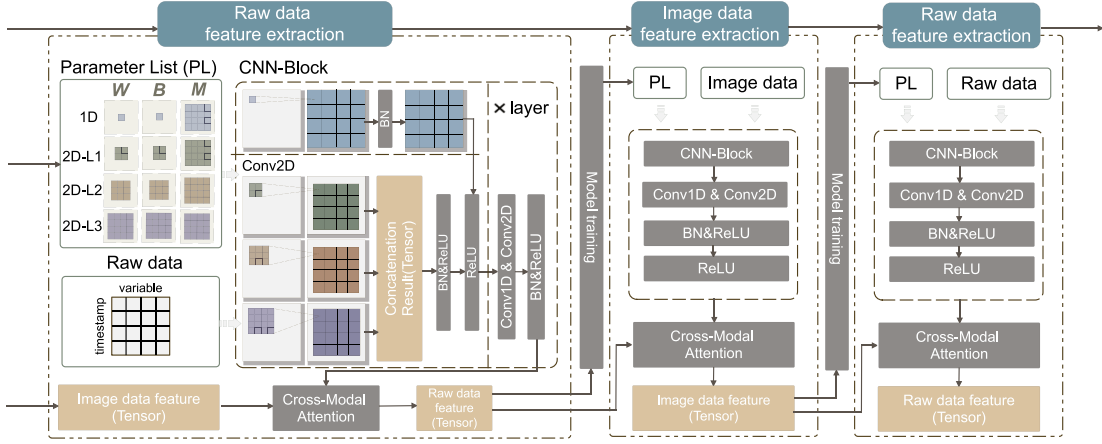


Fig. 3. Multimodal feature extraction.

effectively, the receptive field design must be both flexible and efficient. Using convolution kernels of even sizes enables coverage of receptive fields at various scales [41].

The receptive field (denoted as RF) is calculated as $RF = p(1) + p(2) + p(3) - 2$. For example, if the kernel size of the first convolutional layer p_1 is 3 and the second layer p_2 is 5, the receptive field after a 1D convolution is $RF = 3 + 5 + 1 - 2 = 7$. Similarly, after a 2D convolution, the receptive field becomes $RF = 3 + 5 + 2 - 2 = 8$. Extending this concept, ensuring that $p(1) + p(2)$ covers all even numbers smaller than n enables the network to achieve receptive fields of all sizes less than n .

To maximize coverage, the network uses sets of prime numbers smaller than n as convolution kernel sizes for each layer. This allows processing of receptive fields at different scales, avoiding limitations in local feature extraction and enhancing the network's ability to capture multi-scale information. In the final layer, two convolution kernels— 1×1 and 2×2 —are employed to optimize receptive field coverage. The 1×1 kernel captures features at the smallest scale, while the 2×2 kernel captures slightly larger-scale features. This ensures complete coverage of all receptive field sizes, preventing information loss due to incomplete receptive fields and enhancing the network's robustness in feature extraction.

CNN-Block and Multimodal attention. Based on the defined parameter list, different convolution kernel sizes are determined for the model. Each convolution layer is responsible for processing different temporal windows, allowing the model to extract information across multiple time scales as shown in Equation (9).

$$F_p = \text{Conv}_{k_p \times k_p}(X),$$

$$F_{\text{concat}} = \text{Concat}(F_1, F_2, \dots, F_p), \quad (9)$$

where k_p is the p^{th} kernel size in each layer, and $\text{Conv}_{k_p \times k_p}(X)$ denotes the convolution operation applied to the input X with a kernel size of $k_p \times k_p$. This operation extracts local patterns and features within a specific temporal window defined by k_p . Concat is used to concatenate the features with different kernel sizes along the channel dimension, forming a consolidated feature representation F_{concat} .

To preserve information in deeper networks, multimodal feature extraction adopts a residual connection design. Through skip connections, the input data is directly added to the output, ensuring original information is passed to subsequent layers and mitigating the gradient diminishing problem. In each layer, the input passes through both the main network and a 1×1 convolution layer, which adjusts the channel dimensions to match the extracted features. The residual connection output, $\text{Conv}_{1 \times 1}(X)$, is the transformed input, while the combined output, F_{output} , is the sum of the concatenated features F_{concat} and the residual connection $\text{Conv}_{1 \times 1}(X)$, computed as follows.

$$F_{\text{output}} = F_{\text{concat}} \oplus \text{Conv}_{1 \times 1}(X) \quad (10)$$

To enable each input type to focus on relevant features from the other, we calculate cross-type attention weights. For the numerical input, we compute the query Q_{num} , key K_{num} , and value V_{num} as follows: $Q_{\text{num}} = F_{\text{num}} \mathbf{W}^Q$, $K_{\text{num}} = F_{\text{num}} \mathbf{W}^K$, and $V_{\text{num}} = F_{\text{num}} \mathbf{W}^V$. For the image input, we compute the query Q_{img} , key K_{img} , and value V_{img} as follows: $Q_{\text{img}} = F_{\text{img}} \mathbf{W}^Q$, $K_{\text{img}} = F_{\text{img}} \mathbf{W}^K$, and $V_{\text{img}} = F_{\text{img}} \mathbf{W}^V$. Here, $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$ are weight matrices learned during training. Next, we compute attention weights to let the numerical data attend to the image data, extracting relevant features:

$$\text{Attention}_{\text{num} \rightarrow \text{img}} = \text{softmax} \left(\frac{Q_{\text{num}} K_{\text{img}}^T}{\sqrt{d}} \right),$$

$$F_{\text{num} \rightarrow \text{img}} = \text{Attention}_{\text{num} \rightarrow \text{img}} \oplus V_{\text{img}}, \quad (11)$$

where Q_{num} is multiplied by K_{img}^T (transpose of K_{img}) to compute similarity scores, scaled by $\sqrt{d}$, where d is the feature dimension. The result is passed through a softmax function to calculate attention weights $\text{Attention}_{\text{num} \rightarrow \text{img}}$. These weights are then applied to V_{img} to produce updated features $F_{\text{num} \rightarrow \text{img}}$.

Finally, we reverse the process to let the image data attend to the numerical data:

$$\text{Attention}_{\text{img} \rightarrow \text{num}} = \text{softmax} \left(\frac{Q_{\text{img}} K_{\text{num}}^T}{\sqrt{d}} \right),$$

$$F_{\text{img} \rightarrow \text{num}} = \text{Attention}_{\text{img} \rightarrow \text{num}} \oplus V_{\text{num}}, \quad (12)$$

where Q_{img} is multiplied by K_{num}^T (transpose of K_{num}) to compute similarity scores, scaled by $\sqrt{d}$. The `softmax` function converts the scores into attention weights, $\text{Attention}_{\text{img} \rightarrow \text{num}}$, which are applied to V_{num} to produce updated features $F_{\text{img} \rightarrow \text{num}}$. The above process allows each input type to focus on complementary information from the other, enhancing feature representation.

Parameter sharing: Parameter sharing across different data types reduces the number of parameters and improves training efficiency. The shared parameters include the weights $\mathbf{W}$, biases $\mathbf{B}$, and masks $\mathbf{M}$ for each convolution kernel in the CNN-Block. During each epoch, the model trains sequentially on numerical data and image data, initializing parameters from the previous training iteration. The process is defined as follows:

$$\begin{aligned} N_b &= f_b(\mathbf{X}_n, (\mathbf{W}_b, \mathbf{B}_b, \mathbf{M}_b)) \\ &\leftarrow f_{b-1}(\mathbf{X}_i, (\mathbf{W}_{b-1}, \mathbf{B}_{b-1}, \mathbf{M}_{b-1})), \\ I_b &= f_b(\mathbf{X}_i, (\mathbf{W}_b, \mathbf{B}_b, \mathbf{M}_b)), \end{aligned} \quad (13)$$

where $\mathbf{X}_n$ and $\mathbf{X}_i$ are the numerical and image data, respectively; b denotes the batch index in an epoch; $\mathbf{W}$, $\mathbf{B}$, and $\mathbf{M}$ are the shared weight, bias, and mask parameters of the CNN-Block; N_b and I_b are the embeddings generated for the numerical and image data, respectively; and f_b represents the above multimodal feature extraction process. For each batch b , the numerical data $\mathbf{X}_n$ is processed first using the parameters $\mathbf{W}_b$, $\mathbf{B}_b$, and $\mathbf{M}_b$ obtained from the image data in the previous batch $b-1$. The image data $\mathbf{X}_i$ is then processed using the parameters updated after training on $\mathbf{X}_n$ in the current batch. This sequential training captures correlations between numerical and image data, enhancing the model's ability to learn effectively from multimodal inputs.

Parameter sharing not only significantly accelerates gradient convergence but also offers multiple additional benefits. Theoretically, consider gradients from two modalities denoted as $\mathbf{g}_n$ and $\mathbf{g}_i$. The gradient update for the shared parameters is a weighted linear combination:

$$\Delta\theta_s = \lambda \mathbf{g}_n + (1 - \lambda) \mathbf{g}_i, \quad (14)$$

where $\lambda \in [0, 1]$ is a weighting factor that controls the relative contribution of the two gradients to the final parameter update. The magnitude $\|\Delta\theta_s\|$ denotes the overall step size of the update, determined by the direction and scale of the combined gradients, and can be calculated as: $\|\Delta\theta_s\| =$

$$\sqrt{\lambda^2 \|\mathbf{g}_n\|^2 + (1 - \lambda)^2 \|\mathbf{g}_i\|^2 + 2\lambda(1 - \lambda) \|\mathbf{g}_n\| \|\mathbf{g}_i\| \cos \theta}, \quad (15)$$

where $\cos \theta = \frac{\mathbf{g}_n \cdot \mathbf{g}_i}{\|\mathbf{g}_n\| \|\mathbf{g}_i\|}$, measures the alignment between the two gradient directions. When $\cos \theta$ approaches to 1 (indicating high directional similarity), the combined gradient magnitude is maximized, resulting in more effective updates and significantly faster convergence.

This mechanism enables parameter sharing to promote information fusion between different modalities, allowing the model to learn more generalizable and robust feature representations, thereby improving generalization and noise resilience. Furthermore, the weighted combination of gradients inherently

suppresses noise: when one modality's gradient is noisy, the other can partially offset the disturbance, enhancing training stability and model robustness. Additionally, by reusing the same parameter set, parameter sharing drastically reduces the total number of model parameters, lowering computational and storage costs, while simplifying the model architecture and training process, thus improving overall the training efficiency.

2) *Multimodal Feature Fusion:* It integrates features from multiple modalities to enhance representation for classification tasks. Multimodal feature fusion includes five key steps: (i) concatenation, (ii) depthwise separable convolution, (iii) layer normalization, and (iv) gated Feedforward Network.

Concatenation: The features extracted from different modalities are concatenated to form a combined feature vector $F_{\text{concat}} = [F_{\text{img}}; F_{\text{num}}]$.

Depthwise separable convolution: The concatenated feature vector F_{concat} is processed using a depthwise separable convolution. This encompasses two sequential steps: (i) a depthwise convolution extracts spatial or temporal relationships within individual feature channels, and (ii) a point wise convolution integrates information across all channels, enhancing feature interaction and representation. The resulting output F_{out} is computed as follows:

$$F_{\text{out}} = \text{PointwiseConv}(\text{DepthwiseConv}(F_{\text{concat}})) \quad (16)$$

Splitting standard convolution into these two steps significantly reduces computational complexity by minimizing cross-channel operations, while preserving the richness of extracted features through efficient intra-channel pattern extraction and cross-channel integration.

Layer normalization: Layer normalization stabilizes and accelerates training by normalizing the input across features for each data sample, resulting in F_{norm} .

Gated Feedforward Network: The Gated Feedforward Network (GDFN) achieves refined fusion of multimodal features through dynamic channel weight. Given the input feature F_{norm} , it is first projected into a higher-dimensional space through a pointwise convolution. A subsequent depthwise convolution is applied to capture intra-channel dependencies, producing intermediate features F_v . The feature F_v is then split along the channel dimension into two parts, F_{v_1} and F_{v_2} . A gating mechanism is applied as follows: $F_w = \text{GELU}(F_{v_1}) \odot F_{v_2}$, where $\odot$ denotes element-wise multiplication. This operation adaptively reweights the feature channels, suppressing modality-specific redundancy while emphasizing informative signals. Finally, a 1×1 convolution reduces dimensionality: $\mathbf{Z} = \text{Conv}_{1 \times 1}(F_w)$. This process enables GDFN to perform fine-grained modeling and learnable weighting of multimodal features, effectively enhancing the discriminative power and robustness of the representation. The fused vector $\mathbf{Z}$ is then passed to the classifier for anomaly detection.

The classifier begins with global average pooling to compute the global averages of the input features, reducing their dimensionality and generating a compact representation. This low-dimensional representation is further processed by a fully

Algorithm 1: Multimodal-CNN Model.

Input: The raw data: $input^n$, the MV-MTF data: $input^i$, parameter list: $parameter_list$, the weight, bias and mask of each layer: W , B and M , the layers of mode: $layers$, the number of epoch: $epoch_num$

Output: The feature extraction model: $model$

```

1 foreach  $i \in range(layers)$  do
2    $using\_relu \leftarrow (i == layer - 1);$ 
3    $layer \leftarrow$ 
     Build_Layer( $layer\_parameters, using\_relu$ );
     add  $layer$  to  $layer\_list$ ;
4   initial CNN-Block using  $layer\_list$ ;
5 end
6 foreach  $epoch \in range(epoch\_num)$  do
7   foreach  $j \in range(batch\_num)$  do
8     foreach  $CNN \in CNN\_Block$  do
9        $output^n \leftarrow CNN(input^n, (W, B, M));$ 
10       $output^{n'} \leftarrow ReLu(add(input^n, output^n));$ 
11       $input^n \leftarrow output^n$ ;
12    end
13     $F_{num} \leftarrow FullConnect(Pooling(output^{n'}))$ 
14    foreach  $CNN \in range(CNN\_Block\_num)$  do
15       $output^i \leftarrow CNN(input^i, (W, B, M));$ 
16       $output^{i'} \leftarrow ReLu(add(input^i, output^i));$ 
17       $input^i \leftarrow output^i$ ;
18    end
19     $F_{img} \leftarrow FullConnect(Pooling(output^{i'}))$ ;
20    initial  $Q, K, V$  for multimodal attention;
21     $F_{num \rightarrow img} \leftarrow Attention_{num \rightarrow img} \oplus V_{img}$ ;
22     $F_{img \rightarrow num} \leftarrow Attention_{img \rightarrow num} \oplus V_{num}$ ;
23     $F_{concat} \leftarrow Fusion(F_{num \rightarrow img}, F_{img \rightarrow num})$ ;
24    Adam();
25 end

```

connected layer, which produces the final anomaly detection results.

3) *Model Construction and Training:* Algorithm 1 outlines the construction and training process of the Multimodal-CNN.

Construction: The algorithm begins by defining input parameters, including the original time series data (denoted as $input^n$), image time series data (denoted as $input^i$), model parameters (denoted as $parameter_list$), the weight, bias and mask of each layer (denoted as W , B , and M), the number of layers (denoted as $layers$), and the number of training epochs (denoted as $epoch_num$).

Next, convolutional layers are constructed iteratively. For each layer, the algorithm checks if it is the final layer and determines whether to apply the ReLU activation function accordingly (lines 2–3). The Build_Layer function is then used to create the current layer, which is appended to the $layer_list$. Finally, all layers are combined into a sequential CNN model named *CNN-Block* (lines 4–5).

Training: During the training phase, the algorithm iterates through each training epoch. For each batch, it processes the original time series data using the convolutional blocks. The output $output^n$ from each block is added to the original input, followed by applying the ReLU function to generate a new input $output^n$ (lines 7–11).

Second, the algorithm applies a fully connected layer and pooling operation to $output^n$, producing the feature F_{num} (line 12). Similarly, the image time series data is processed, generating $output^i$, which passes through a fully connected layer and pooling operation to derive the feature F_{img} (lines 14–16).

Next, We initialize the parameters Q , K , and V for the multimodal attention mechanism, which is then employed to capture and fuse inter-modal relationships, producing updated features F_{num} and F_{img} (lines 18–21). These features are fused into a combined feature F_{concat} , and the model is optimized using the Adam optimizer [23] with a learning rate adjustment function (lines 22–23).

Finally, the algorithm returns the trained MultiModal Feature Extraction model $model$. This enables effective extraction and fusion of features from multiple modalities, enhancing the model's overall performance.

C. Anomaly Explainer

The explainer interprets the results of anomaly detection models, as illustrated in the lower right corner of Fig. 2. It includes the Kernel Explainer and Gradient Explainer [1], which are used to interpret numerical and image data, respectively. Kernel methods and gradient methods are employed to generate SHAP values (s^K and s^G), which evaluate each variable's contribution to anomaly detection. s^K represents the direct impact of the current variable's raw data on anomaly detection results, while s^G captures the influence of the current variable's MV-MTF data on the current variable. During anomaly evaluation, s^K and s^G are combined to identify the variables that significantly contribute to anomalies. For the v^{th} variable, its SHAP value s_v is computed as follows:

$$s_v = \omega \times s_v^K + (1 - \omega) \times \frac{\sum_{j=1}^c s_j^K \times s_v^G}{c}, \quad (17)$$

where c is the total number of variables, ω is a weight parameter balancing the contributions of s^K and s^G , s_v^K is the SHAP value reflecting the direct impact of the v^{th} variable, and s_v^G is the impact of other variables on the v^{th} variable. Equation (17) exhibits Bounded Controllability and Top-k Ranking Plateau properties.

Let the fused attribution for variable v be $s_v(\omega) = \omega s_v^K + (1 - \omega) \gamma s_v^G$ with $\gamma = \frac{1}{c} \sum_{j=1}^c s_j^K$. Collecting terms in ω yields $s_v(\omega) = a_v \omega + b_v$ with $a_v := s_v^K - \gamma s_v^G$ and $b_v := \gamma s_v^G$.

Lemma 1 (Bounded controllability). Given $s_v(\omega) = a_v \omega + b_v$ and let $L := \|a\|_\infty = \max_v |a_v|$. Then for any v and any $\omega, \omega' \in [0, 1]$, $|s_v(\omega') - s_v(\omega)| \leq |a_v| |\omega' - \omega| \leq L |\omega' - \omega|$. Hence each s_v is L -Lipschitz in ω , ensuring bounded sensitivity and controllability with respect to the weighting parameter.

Proof. For any perturbation δ with $\omega + \delta \in [0, 1]$, since $s_v(\omega) = a_v\omega + b_v$, we have

$$\begin{aligned} s_v(\omega + \delta) - s_v(\omega) &= a_v \delta \\ \Rightarrow |s_v(\omega + \delta) - s_v(\omega)| &= |a_v| |\delta| \leq \underbrace{\max_v |a_v|}_{=\|a\|_\infty=L} |\delta|. \end{aligned} \quad (18)$$

Equivalently, for any $\omega, \omega' \in [0, 1]$,

$$|s_v(\omega') - s_v(\omega)| = |a_v(\omega' - \omega)| \leq L |\omega' - \omega|. \quad (19)$$

Hence, each s_v is L -Lipschitz in ω , establishing bounded controllability. ■

As $L = \max_v |a_v^K|$ and $a_v = s_v^K - \gamma s_v^G$, we can get that $L \leq \max_v |s_v^K| + \gamma \cdot \max_v |s_v^G|$. In binary classification, it is known that $\max_v |s_v^K| = 1$, and for anomalous data the MV-MTF values are very small, which leads to $|s_v^G| \approx 0$. Therefore, we can derive that $L \leq 1$. Hence, for any ω, ω' , it holds that $|s_v(\omega') - s_v(\omega)| \leq |\omega' - \omega|$. This indicates that as ω varies, the contribution of variable v is constrained by a uniform upper bound, and its ranking is therefore stable.

Given the ranked contribution list $C(\omega) = \{s_i(\omega)\}_{i=1}^n$, where n is the number of variables and $C(\omega)$ is in descending order of $s_i(\omega)$, the Top- k set is defined as k variables with highest $s_i(\omega)$, i.e., $\text{Top-}k(\omega) = \{s_1(\omega), \dots, s_k(\omega)\}$.

Lemma 2 (Top- k ranking plateau). For any two variables u and v , the pairwise contribution difference is defined as $\Delta_{uv}(\omega) := s_u(\omega) - s_v(\omega) = A_{uv}\omega + B_{uv}$, where $A_{uv} = a_u - a_v$ and $B_{uv} = b_u - b_v$. Then, with $S := \max_{u,v} |A_{uv}|$, the Top- k set remains locally invariant within radius $|\delta| = (s_k(\omega) - s_{k+1}(\omega))/S$, i.e., $\text{Top-}k(\omega + \delta) \equiv \text{Top-}k(\omega)$.

Proof. For any $u \in \text{Top-}k$ and $v \notin \text{Top-}k$, given the offset value δ of ω , then

$$\begin{aligned} \Delta_{uv}(\omega + \delta) &= A_{uv}(\omega + \delta) + B_{uv} \\ &\geq \Delta_{uv}(\omega) - |A_{uv}| |\delta| \\ &\geq (s_k(\omega) - s_{k+1}(\omega)) - |A_{uv}| |\delta|. \end{aligned} \quad (20)$$

Only when $\Delta_{uv}(\omega + \delta) > 0$, we have $\text{Top-}k(\omega + \delta) = \text{Top-}k(\omega)$. To ensure this condition simultaneously across all variable pairs, we set $S := \max_{u,v} |A_{uv}|$, which leads to the sufficient condition $|\delta| < (s_k(\omega) - s_{k+1}(\omega))/S$. Here, $s_k(\omega) - s_{k+1}(\omega)$ represents the minimal margin between Top- k and non-Top- k variables. Under this condition, the Top- k set remains unchanged. ■

$S := \max_{u,v} |A_{uv}|$ is the same for all ω . Therefore, δ is determined by $s_k(\omega) - s_{k+1}(\omega)$. From $s_v(\omega) = \omega s_v^K + (1 - \omega) \gamma s_v^G$, we can find that increasing ω raises the contribution of the raw data s_v^K , while decreasing ω raises the contribution of the transition MV-MTF data γs_v^G . Under anomalies, the values of the transition structure MV-MTF tend to decrease (due to the low probability of anomalous transitions), which in turn drives the contribution s_v^G close to zero, thereby leading to a very small $s_k(\omega) - s_{k+1}(\omega)$ value. The value of $s_k(\omega) - s_{k+1}(\omega)$ increases only when ω grows, which means

the weight of the raw data contribution is increased. According to $|\delta| < \frac{s_k(\omega) - s_{k+1}(\omega)}{S}$, it follows that as $s_k(\omega) - s_{k+1}(\omega)$ increases, Δ also grows, thereby widening the plateau region.

This weighting scheme effectively balances the contributions of raw numerical features and MV-MTF modalities, ensuring anomaly detection primarily relies on reliable raw data while moderately integrating the complex structural information captured by MV-MTF.

Based on Equation (17), we rank variables by their contribution to anomalies. Variables with higher s_v contribute more, while those with lower s_v have little or no contribution. Identifying the contributing variables is essential. To achieve this, we propose a binary search strategy to efficiently determine a critical point k , defined as follows:

Definition 4 (Critical point). Given a list of variables $\mathcal{L}$ ranked by their contribution to anomalies, k is a critical point such that the first k variables in $\mathcal{L}$ contribute to the anomalies, while the remaining variables do not. $\mathcal{L}_{o,o'}$ denotes a sublist of $\mathcal{L}$ from $\mathcal{L}_o$ to $\mathcal{L}_{o'}$, where $\mathcal{L}_o$ is the o^{th} variable in $\mathcal{L}$.

Example 5. For an anomaly time series with eight variables, we first calculate s_v ($1 \leq v \leq 8$) using 10 and rank the variables by s_v to obtain $\mathcal{L} = \{2, 4, 5, 1, 8, 7, 6, 3\}$. This ranking indicates that variable 2 is ranked first, variable 4 s, and so on.

Next, we apply the binary search strategy. First, we replace the first four variables $\mathcal{L}_{1,4} = \{2, 4, 5, 1\}$ —the first half of $\mathcal{L}$ —with normal sequences and check whether the anomaly remains. Two cases arise: (i) If the anomaly remains, $4 \leq k \leq 8$, and we replace $\mathcal{L}_{4,5}$ —the first half of $\mathcal{L}_{4,8}$ —with normal sequences. (ii) If the anomaly is resolved, $1 \leq k \leq 4$, and we replace $\mathcal{L}_{1,2}$ —the first half of $\mathcal{L}_{1,4}$ —with normal sequences. This process is repeated until k is determined, identifying the first k variables as those contributing to the anomalies.

During anomaly analysis, anomalous data is clustered and evaluated with expert knowledge to determine the nature of each anomaly. For example, in the SMD dataset, anomalies are categorized into the three types: (i) network failure, (ii) hardware damage, and (iii) software service anomaly. New anomalies can be classified into these predefined types and further interpreted in detail.

Finally, by combining the results of anomaly evaluation and analysis, an anomaly report is generated. This report not only provides detailed explanations of anomaly detection results but also enhances the accuracy and actionability of the analysis through evaluation and classification mechanisms. It offers users deeper insights and robust support for decision-making.

V. EXPERIMENTS

A. Experimental Setting

Datasets: We evaluate MOON on six real-world MTS datasets.

- Soil Moisture Active Passive (SMAP) [19]: It consists of soil samples and telemetry data collected by NASA's Mars rover.
- Mars Science Laboratory (MSL) [48]: It is similar to SMAP but corresponds to the sensor and actuator data for the Mars rover itself.

TABLE II
DATASET STATISTICS

DataSet	#Train	#Test	#Entities	#Dimensions
WADI	1048571	172801	1	123
MSL	58317	73729	3	55
PSM	132482	87842	1	26
SMAP	58317	73729	55	25
SWaT	496800	449919	1	51
SMD	708405	708420	28	38

- Secure Water Treatment (SWaT) [32]: It is collected from a real-world water treatment plant with seven days of normal and four days of abnormal operation. This dataset consists of sensor values (water level, flow rate, etc.) and actuator operations (valves and pumps).
- Water Distribution dataset (WADI) [32]: It is an extended dataset of SWAT. WADI features 14 days of normal operation KPIs and two days of attack scenario KPIs.
- Pooled Server Metrics (PSM) [2]: It is collected from multiple eBay application server nodes, anonymized for publication. It includes 26 features related to server machine metrics like CPU utilization and memory, with some localization meta-attributes omitted. The training set spans 13 weeks, followed by eight weeks for testing.
- Server Machine Dataset (SMD) [39]: It is a five-week long dataset of stacked traces of the resource utilizations of 28 machines from a compute cluster. Similar to MSL, we use the nontrivial sequences, specifically the traces for machine-2-1, machine-2-6, and machine-3-7.

The datasets used are publicly available and can be accessed at the following URLs: SMAP¹, MSL², SWaT³, WADI⁴, PSM⁵, and SMD⁶. We summarize the key characteristics of the datasets in Table II. “#Entities” refers to the number of distinct time series, and “#Dimensions” refers to the number of dimensions in each dataset. Each dataset includes a training set without anomaly detection labels and a testing set with labels. For our experiments, we use 6,000 training samples for the WADI dataset and 10,000 for the other five datasets. All datasets use 5,000 testing samples.

Baselines and setting: We compare MOON with eight state-of-the-art reconstruction-based methods: TRANAD [42], OMNIANOMALY [39], MAD_GAN [26], LSTM_AD [20], TIMESNET, ADTRANSFORMER, DCDETECTOR and CATCH [49] and two classification-based methods: HYPERROCKET [11] and ROCKET [10], [40]. We choose the two types of methods as baselines because (i) prediction-based methods are theoretically similar to reconstruction-based methods, with the latter being more mainstream, and (ii) MOON is a classification-based method. Due to the space limitation, please refer to Section VI for more details about baselines. In our experiments, the hyperparameters

are configured as follows: the coefficient α in (6) is set to 0.9, the learning rate is set to 0.001, and the weight ω in (17) is fixed at 0.8.

Performance metrics: We evaluate anomaly detection performance using Precision, Recall, F1 score, and F1_{PA} score. F1_{PA} score is a segment-based F1 score using point adjustment (PA), where a segment is considered abnormal if at least one point within it is detected as abnormal [18], [22]. $F1_{PA} = 2 \cdot \frac{\text{Precision}_{PA} \cdot \text{Recall}_{PA}}{\text{Precision}_{PA} + \text{Recall}_{PA}}$, where Precision_{PA} and Recall_{PA} are the precision and recall calculated using the PA method.

We evaluate the interpretability of anomaly detection using Hit@P%, which measures the proportion of true anomalous dimensions included among the top candidates predicted by the model [30]. Here, P% is the percentage of true anomalous dimensions at each timestamp, defining the number of top predicted candidates considered.

Environment: All methods are executed on a machine with an Intel(R) Core(TM) i9-10900 K CPU, featuring 10 cores and a 3.70 GHz clock speed. The framework is also equipped with an NVIDIA GeForce RTX 3090 graphics card, which has 24 GB of video memory. The source code of MOON are available.⁷

B. Comparison Study

We compare MOON with reconstruction-based methods (TRANAD, OMNIANOMALY, MAD_GAN, LSTM_AD, TIMESNET, ADTRANSFORMER, DCDETECTOR and CATCH) and classification-based methods (HYPERROCKET and ROCKET) in six datasets. Table III presents the results, highlighting the best performance in bold.

Compared with reconstruction-based methods: MOON achieves the highest or near-highest Recall, F1 and F1_{PA} on most datasets, with Precision values almost reaching 1 across all datasets. In contrast, reconstruction-based methods such as TRANAD, OMNIANOMALY, MAD_GAN, CATCH, and LSTM_AD often have lower F1 and F1_{PA} on complex datasets, particularly on the SMAP and SWaT datasets where the F1 are below 0.9. This is because the Moon method improves the ability to identify anomalies in complex multivariate time-series data through modal transformation and deeper feature extraction and fusion techniques. Meanwhile, reconstruction-based methods rely on reconstruction error, which is effective for simple patterns. However, they struggle with multivariate, high-dimensional, and complex dependencies because they cannot fully capture anomaly characteristics, resulting in lower detection performance.

Furthermore, although many reconstruction-based methods generally excel in Recall, detecting most anomalies and thus maintaining a low miss rate, they struggle with Precision, with higher false positive rates. For instance, OMNIANOMALY has a Precision of only 0.7991 on the SMAP dataset, and MAD_GAN has a Precision of only 0.8725 on the PSM dataset, both significantly lower than MOON. This indicates that these methods have difficulties distinguishing between normal and anomalous data, possibly due to reconstruction error inadequately capturing

¹<https://nsidc.org/data/smap/data>

²https://pds-atmospheres.nmsu.edu/data_and_services/atmospheres_data/Mars/Mars.html

³https://itrust.sutd.edu.sg/itrust-labs_datasets/dataset_info/#swat

⁴https://itrust.sutd.edu.sg/itrust-labs_datasets/dataset_info/#wadi

⁵<https://github.com/eBay/RANSynCoders/tree/main/data>

⁶<https://github.com/NetManAIops/OmniAnomaly/tree/master/ServerMachineDataset>

⁷<https://github.com/Syh517/Moon/>

TABLE III
COMPARISON RESULTS

Method	WADI				MSL				PSM			
	Precision	Recall	F1	FIPA	Precision	Recall	F1	FIPA	Precision	Recall	F1	FIPA
TranAD	0.9460	0.9992	0.9719	0.9742	0.8649	0.9831	0.9202	0.9914	0.9494	0.9999	0.9494	0.9784
OmniAnomaly	0.8549	0.9999	0.9218	0.9478	0.7848	0.9999	0.8794	0.9917	0.8816	0.9990	0.8985	0.9522
MAD_GAN	0.9422	0.9596	0.9702	0.9702	0.8516	0.9999	0.9198	0.9801	0.8725	0.9968	0.9287	0.9417
LSTM_AD	0.8953	0.9999	0.9447	0.9686	0.7948	0.9999	0.9023	0.9683	0.9038	0.9999	0.9494	0.9814
TimesNet	0.7973	0.9900	0.8833	0.9948	0.9735	0.9715	0.9725	0.9855	0.9850	1.0000	0.9924	1.0000
ADtransformer	0.7926	0.9620	0.8692	0.9804	0.9802	0.9842	0.9822	0.9920	0.9908	0.9878	0.9893	0.9939
DCdetector	0.7961	0.9144	0.8512	0.9551	0.9739	0.9589	0.9664	0.9790	0.9809	0.8910	0.9338	0.9358
CATCH	0.7968	0.9524	0.8677	0.9754	0.9773	0.8651	0.9178	0.9277	0.9815	0.9745	0.9780	0.9780
HyperRocket	-	-	-	-	0.9771	0.9355	0.9558	0.9691	0.9820	1.0000	0.9909	0.9909
Rocket	0.6923	0.8735	0.9945	0.9945	0.9773	0.9378	0.9571	0.9774	0.9850	1.0000	0.9909	0.9909
Moon	1.0000	0.9912	0.9956	0.9956	0.9772	0.9961	0.9866	0.9980	0.9869	1.0000	0.9934	1.0000
Method	SMAP				SWaT				SMD			
	Precision	Recall	F1	FIPA	Precision	Recall	F1	FIPA	Precision	Recall	F1	FIPA
TranAD	0.9133	0.9965	0.9531	0.9706	0.9977	0.6878	0.8143	0.9773	0.9072	0.9973	0.9501	0.9981
OmniAnomaly	0.7991	0.9989	0.8883	0.9619	0.9760	0.6956	0.8123	0.9769	0.9881	0.9985	0.9932	0.9983
MAD_GAN	0.8157	0.9999	0.8984	0.9634	0.9593	0.6956	0.8064	0.9770	0.9967	0.9980	0.9973	0.9982
LSTM_AD	0.8139	0.9999	0.8974	0.9702	0.9977	0.6878	0.8143	0.9318	0.9069	0.9973	0.9499	0.9695
TimesNet	0.9107	0.9993	0.9530	0.9997	0.8072	0.9988	0.8928	0.9994	0.9918	0.9989	0.9953	0.9994
ADtransformer	0.9103	0.9939	0.9502	0.9969	0.8074	1.0000	0.8934	1.0000	0.9930	0.9951	0.9940	0.9944
DCdetector	0.9107	0.9405	0.9253	0.9693	0.8075	0.9978	0.8926	0.9989	0.9918	0.8554	0.9185	0.9219
CATCH	0.9109	0.9785	0.9435	0.9891	0.8077	0.9762	0.8840	0.9880	0.9916	0.9357	0.9628	0.9661
HyperRocket	0.9107	0.9989	0.9528	0.9961	0.7379	0.0907	0.1615	0.1663	0.9933	0.9311	0.9612	0.9612
Rocket	0.9106	1.0000	0.9532	0.9532	0.6936	0.4985	0.5801	0.6653	0.9936	0.9652	0.9792	0.9957
Moon	0.9561	1.0000	0.9776	1.0000	0.8111	0.9787	0.8870	0.9892	0.9914	0.9999	0.9956	0.9956

anomaly characteristics. In contrast, the comprehensive feature extraction and classification techniques of MOON enable accurate anomaly detection across various datasets. In addition, MOON outperforms CATCH in most cases. This is because CATCH transforms time series into the frequency domain, capturing frequency features but potentially losing temporal structure, making it less effective for anomalies that rely on fine-grained temporal context. However, MOON uses raw and enhanced anomaly data to better capture details, achieving more accurate detection.

Compared with classification-based methods: MOON outperforms the classification-based methods in F1-Score on most datasets in all performance metrics. However, classification-based methods often show a clear disparity between Precision and Recall. For instance, in the SMAP dataset, HYPERROCKET has a Precision of only 0.9107, while its Recall reaches 1.0. This result indicates that HYPERROCKET is highly sensitive to anomalies but lacks the ability to accurately distinguish between normal and anomalous data. This occurs because the Rocket method assigns random kernels to each variable, disregarding the correlations between variables. In contrast, Moon enhances the ability to identify anomalies in complex MTS data by capturing local information between variables through modality conversion, showcasing excellent overall performance. Additionally, HyperRocket performs slightly worse than Rocket. This is because HyperRocket emphasizes global modeling and cross-variable dynamics, which benefits sequence-level classification [50] but may overlook subtle point-wise anomalies. In contrast, Rocket uses random convolutional kernels to extract local features, making it more effective for detecting anomalies at individual time points. Classification-based methods generally

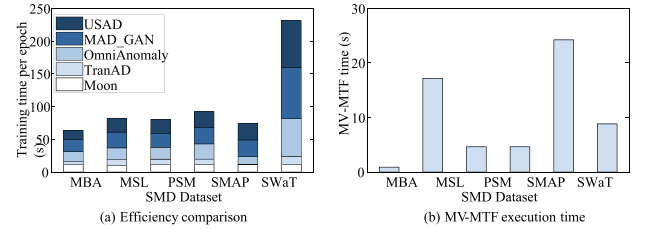


Fig. 4. Training efficiency study.

achieve higher F1-Score than reconstruction-based methods on most datasets. This is due to classification methods not relying on anomaly scores, allowing them to more effectively capture abnormal patterns in the data.

C. Efficiency Study

Training efficiency study: To evaluate the efficiency, we compare the running time per epoch of MOON with that of reconstruction-based methods. HYPERROCKET and ROCKET, which use predefined feature extraction instead of deep neural networks, do not require training. Hence, their epoch running times cannot be measured and are excluded from the comparison. The results shown in the Fig. 4(a) indicate that MOON's running time per epoch is comparable to that of TRANAD and significantly lower than that of other methods, especially on SMD datasets. This result stems primarily from two factors. First, MOON's efficiency is primarily due to its use of parameter sharing, which reduces redundant computations, and its CNN-based feature extraction module, which accelerates the feature extraction process and further enhances overall efficiency.

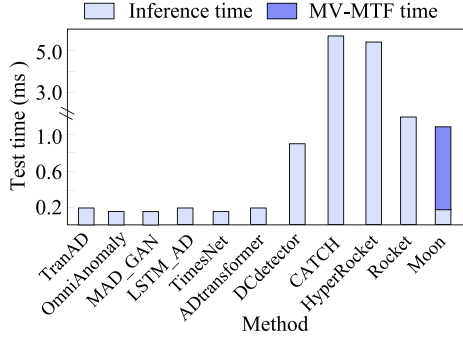


Fig. 5. The test time (ms) of each time series data point.

Second, TRANAD is a lightweight method which contributes to its fast performance. In contrast, other reconstruction-based methods process all sequential windows, making them more computationally intensive.

Fig. 4(b) presents the execution time of MV-MTF technology in MOON across all six datasets. The results demonstrate that the execution time of MV-MTF increases with the number of variables. For instance, it takes only a few seconds when the number of variables is less than 40, as seen in PSM (26 variables), SMAP (25 variables), and SMD (38 variables). However, the execution time increases significantly when the number of variables exceeds 50, as demonstrated by the MSL (55 variables), SWaT (51 variables) and WADI (123 variables) datasets. However, MV-MTF transformation is performed once during the entire training process. We have optimized its time complexity to $O(n)$ (see Section IV-A), making its time consumption negligible compared to overall training time. In real-time applications, reconstruction- and prediction-based methods compare with original data, while Moon uses modal conversion. Both have $O(n)$ complexity, proportional to window size and number of variables, resulting in minimal efficiency differences.

Inference efficiency study: To demonstrate the practical feasibility of our approach in real-time streaming settings, we further report the online testing time, as shown in Fig. 5. It is important to note that our method is not designed for real-time anomaly detection in streaming settings. Therefore, we first calculate the total test time, then divide by the number of samples to get the inference time per TS point. Experimental results show that the anomaly detection time (i.e., 0.2 ms) of our classification-based model is comparable to that of existing unsupervised methods. Although the MV-MTF transformation introduces an additional 1.5 ms per time series data point, the overall time remains more efficient than traditional supervised classifiers Rocket and HyperRocket, as our model avoids complex high-dimensional computations during inference.

D. Ablation Study

Ablation study on multimodal-CNN: We evaluate the effectiveness of the proposed Multimodal-CNN by comparing it to ResNet, LSTM, and CNN models across four metrics: Precision, Recall, F1, and F1_{PA}, on six datasets. Fig. 6 presents the comparative results, demonstrating that Multimodal-CNN consistently

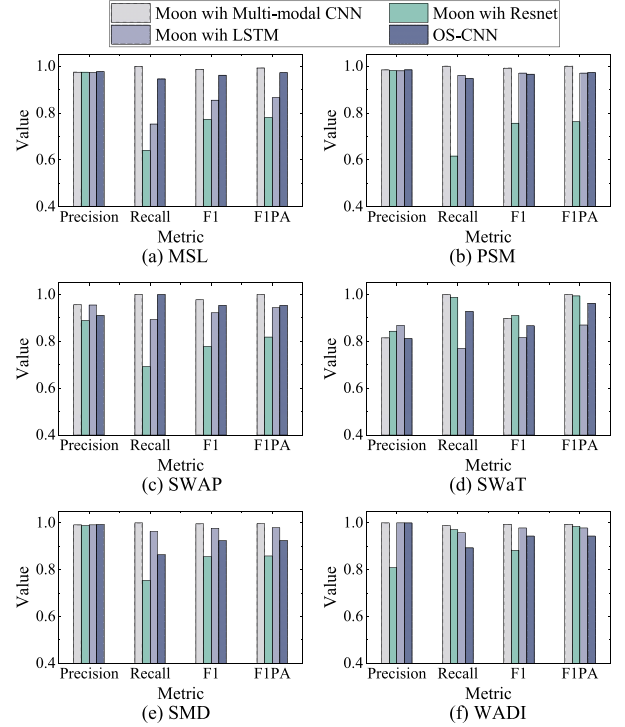


Fig. 6. Ablation Study on Multimodal-CNN of Moon.

outperforms both ResNet, LSTM and CNN across all evaluation metrics, with especially notable improvements over ResNet. This arises from Multimodal-CNN's use of convolutional kernels with varying sizes, enabling it to capture receptive fields at different scales and model both local and global features effectively. In contrast, ResNet's fixed receptive field limits its ability to fully capture the range of data features.

Furthermore, Multimodal-CNN demonstrates a clear advantage over LSTM and CNN in handling high-dimensional MTS data, such as MSL with 55 variables and WADI with 123 variables. This advantage stems from its ability to effectively capture complex inter-variable relationships using a MV-MTF and to enhance key information extraction through a cross-modal attention mechanism. These features enable Multimodal-CNN to better integrate multimodal information and model dependencies between variables. In contrast, LSTM and CNN lack the capability to effectively capture inter-variable dependencies. As a result, they often overlook interactions and relationships between variables, leading to the loss of critical information.

Ablation study on four key components: We evaluate the effectiveness of three key components—modality conversion, feature fusion, parameter sharing, and multimodal attention—through ablation experiments on the PSM and SMD datasets. Table IV presents the results, with bold values indicating the best performance.

As observed, omitting any component leads to a performance drop, particularly in Recall, F1, and F1_{PA}. On the PSM dataset, removing the feature fusion module reduces Recall, F1, and F1_{PA} by 0.0530, 0.0282, and 0.0277, respectively. This is because simple data concatenation fails to capture the relationships between

TABLE IV
ABLATION STUDY ON MODALITY CONVERSION, FEATURE FUSION, AND
PARAMETER SHARING OF MOON

Method	PSM			
	Precision	Recall	F1	F1 _{PA}
Moon	0.9869	1.0000	0.9934	1.0000
w/o modality conversion				
— only raw data	0.9829	0.9470	0.9652	0.9723
— only MV-MTF data	0.9819	0.9979	0.9898	0.9898
— replace MV-MTF with GAF	0.9790	0.8584	0.9148	0.9148
w/o feature fusion	0.9817	0.9837	0.9827	0.9858
w/o parameter sharing	0.9812	0.9613	0.9711	0.9711
w/o MultiModel attention	0.9806	0.8268	0.8972	0.9022
Method	SMD			
	Precision	Recall	F1	F1 _{PA}
Moon	0.9914	0.9999	0.9956	0.9956
w/o modality conversion				
— only raw data	0.9829	0.9470	0.9652	0.9723
— only MV-MTF data	0.9905	0.9968	0.9942	0.9965
— replace MV-MTF with GAF	0.9910	0.7568	0.8597	0.8788
w/o feature fusion	0.9912	0.6131	0.7578	0.7602
w/o parameter sharing	0.9916	0.7822	0.8745	0.9956
w/o MultiModel attention	0.9932	0.7554	0.8582	0.8582

modalities, resulting in decreased performance. On the SMD dataset, omitting modality transformation (i.e., only raw data) causes a significant drop in Recall, F1, and F1_{PA}, decreasing by 0.1349, 0.0688, and 0.0690, respectively. This highlights the critical role of modality transformation in capturing local inter-variable relationships and improving performance. Only the MV-MTF data or replacing MV-MTF with Gramian Angular Fields (GAF) method still lead to a decline in performance, particularly on the PSM dataset. This is because MV-MTF is specifically designed to capture temporal dependencies and inter-variable relationships in time series. This enables more effective modeling of complex dynamic behaviors and significantly enhances the distinguishability between normal and anomalous patterns. In contrast, GAF primarily relies on static spatial or temporal mappings and struggles to uncover such rich structural information. In addition, the absence of parameter sharing leads to a notable performance decline, emphasizing its importance in integrating information across modalities to enhance feature extraction and overall model performance. Finally, the ablation of the multimodal attention module results in a substantial performance degradation, with Recall, F1, and F1_{PA} dropping to 0.2444, 0.1374, and 0.1374, respectively. This highlights the pivotal role of multimodal attention in capturing cross-modal correlations and complementary information, thereby improving the model's ability to detect abnormal patterns.

E. Parameter Study

Parameter study on α : We conduct a parameter study on the weight parameter α (cf. Equation 5), using the PSM, SMAP, and SMD datasets. As shown in Fig. 8, α exhibits a similar trend. When α is set to 0.1, the performance is notably poor because this setting reduces the transition probabilities between timestamps, leading to insufficient information capture from individual time dimensions and consequently, lower performance. As α increases, performance generally improves. However, the

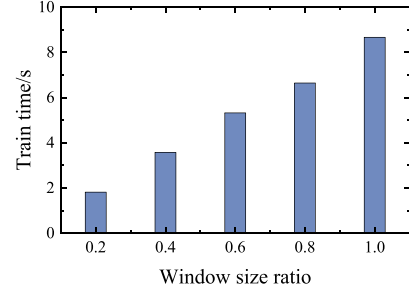


Fig. 7. The efficiency of different window size.

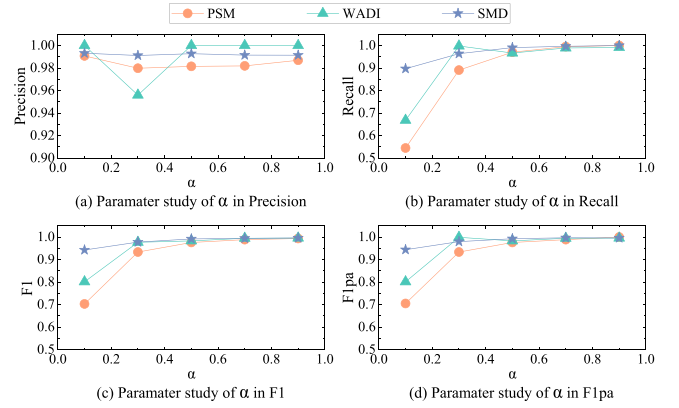


Fig. 8. Parameter study of α .

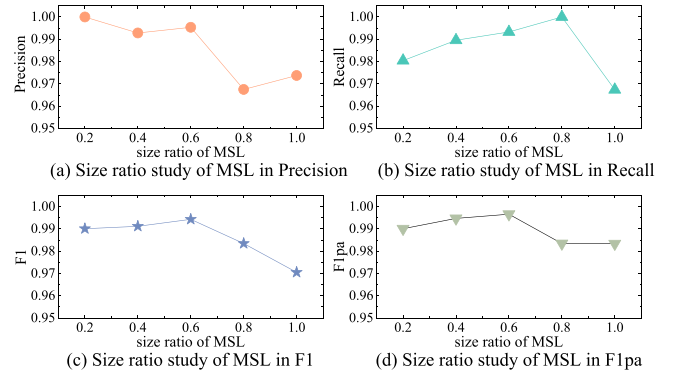


Fig. 9. Parameter study on historical time-step length in performance.

optimal α value for achieving the best performance varies across datasets due to differences in variable dependencies.

Parameter study on historical window sizes: In addition, we conduct a sensitivity analysis on different historical window sizes on MSL dataset to evaluate the robustness of our method. The results presented in Fig. 9 demonstrate that the model achieves optimal performance—in terms of both F1 and F1_{PA} scores—when the window size is set to 0.6. This is because a smaller window may fail to capture the distinction between normal and abnormal patterns, while a larger window tends to obscure critical local anomaly signals.

From an efficiency perspective, smaller window sizes result in faster computation, as illustrated in Fig. 7. This is because

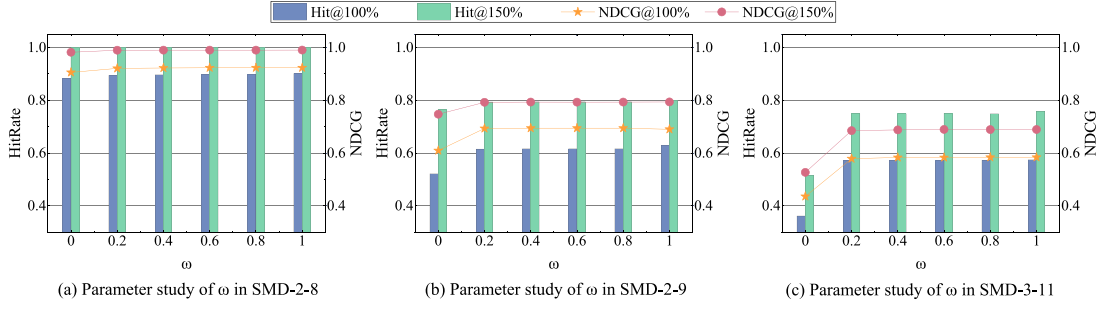


Fig. 10. The weight parameter balancing in the interpretability contribution equation.

TABLE V
ANOMALY INTERPRETATION PERFORMANCE

Method	Hit@100%	Hit@150%	NDCG@100%	NDCG@150%
TranAD	0.4905	0.6634	0.5167	0.6105
OmniAnomaly	0.4873	0.6119	0.5076	0.5831
MAD_GAN	0.4607	0.6598	0.4158	0.5345
LSTM_AD	0.4761	0.6419	0.4474	0.5622
Moon	0.5209	0.7034	0.5294	0.6283

both the transformation overhead of MV-MTF and the overall model complexity increase with the growth of window size.

Parameter study of ω : We perform a parameter study of the fusion weight ω in Eq. (17) over $[0, 1]$ on three SMD subsets stratified by interpretability ground-truth length—long (SMD-2-8), medium (SMD-2-9), and short (SMD-3-11). The results in Fig. 10 yield two consistent findings, aligned with Lemmas 1 and 2: (i) performance varies slightly with ω , fluctuations occur mainly for $\omega \in [0, 0.4]$; and (ii) there exists a broad *plateau* (i.e., $\omega \in [0.6, 1.0]$) on which the variable-interpretability ranking remains essentially unchanged. These patterns persist across all three length regimes, indicating robustness both to the choice of ω and to the ground-truth duration. Therefore, we fix $\omega = 0.8$ as a robust default.

F. Performance of Anomaly Interpretation

We evaluate the anomaly interpretability by using $\text{Hit@P\%} = \frac{\text{Hit@P\%}}{|GT_t|}$, where GT_t is the ground truth array of dimensions that contribute to the anomaly and $|GT_t|$ is its length. Here, $\text{Hit@P\%}$ [39] represents the number of ground truth dimensions in the top $P\% \times |GT_t|$ of the AS list, where P is set to 100 and 150. For example, for an anomaly in a time series with 5 variables, where the ground truth GT_t is $\{2, 3\}$ and the AS list generated by MOON is $\{2, 1, 3, 5, 4\}$, the $\text{Hit@100\%}$ is 50% since only one of the top two variables in AS matches GT_t . The $\text{Hit@150\%}$ is 100%, as both variables in GT_t are among the top three in the AS list. We also use metric (NDCG) [21] to measure the ranking quality of anomaly interpretation. Similar to $\text{HitRate@P\%}$, $\text{NDCG@P\%}$ considers the top $P\% \times |GT_t|$ variables, but it further emphasizes the positions of the ground truth dimensions in the ranking. We evaluate MOON exclusively on the SMD dataset, as other datasets lack interpretation labels.

As shown in Table V, MOON achieves superior anomaly interpretation accuracy, with a $\text{Hit@100\%}$ of 0.5294 and

a $\text{Hit@150\%}$ of 0.7034. In contrast, the state-of-the-art reconstruction-based model TRANAD performs significantly worse on the same dataset, with a $\text{Hit@100\%}$ of 0.4905 and a $\text{Hit@150\%}$ of 0.6634. Reconstruction-based methods such as TRANAD rely on thresholding to identify anomalous variables, making their performance highly dependent on detection accuracy and often overlooking the model’s influence on individual variables. In contrast, MOON focuses on explaining the model’s internal mechanisms rather than merely interpreting its outputs. Moreover, Moon achieves superior performance on the $\text{NDCG@P\%}$ metric, as it emphasizes ranking order rather than simply detecting whether a target is hit. By integrating a scoring mechanism to filter out unreliable explanations, Moon provides more precise interpretations, thereby enhancing both the interpretability and reliability of anomaly detection.

G. Case Study

To validate the performance of our method in real-world scenarios, we conduct a case study using the SMD dataset. We selected the SMD 1–4 subset and focus our analysis on the time interval $[370, 430]$. Within this interval, the dataset labels the period $[385, 417]$ as anomalous. It is important to note that in SMD, anomaly labels are assigned at the time-point level: if any variable is anomalous at a given timestamp, all variables at this timestamp are marked as anomalous. However, based on the dataset’s provided interpretable labels, we confirm that the true anomalous variables during this period are variables 12, 15, and 16, corresponding to `disk_svc`, `disk_wa`, and `disk_wb`, which represent the average service time per disk request, the I/O wait time, and the write-back time, respectively. As shown in Fig. 11, the top three subplots display the time-series curves of these anomalous variables, with red shaded regions indicating the actual anomalous intervals. For comparison, we also include several variables (e.g., 19, 20, 21) that remained normal during the same period.

Interpretable anomaly detection must not only identify anomalous timestamps but also accurately locate the responsible variables. Once an anomaly is detected, we compute the contribution of each variable by combining the MV-MTF representation with the original TS data. To assess the performance of Moon’s explainer, we visualize the contribution results in Fig. 12. As observed, Moon’s explainer correctly identifies 2 out of the 3 ground-truth anomalous variables within its

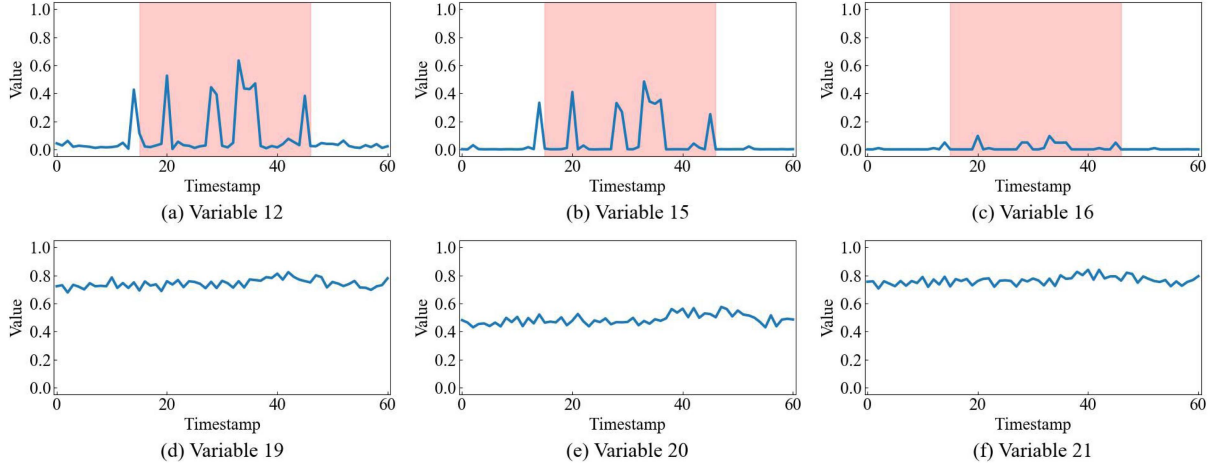


Fig. 11. Time series plots of anomalous variables (12, 15, 16) and partial normal variables (19, 20, 21) in time interval [370, 430] of the SMD 1-4 dataset.

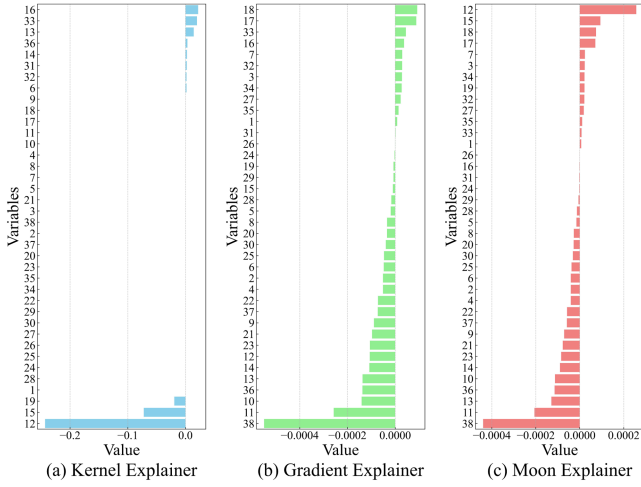


Fig. 12. The relative contribution of variables to the anomalies .

top-3 predictions, yielding a Hit@100% score of 0.667. Finally, engineers can leverage the identified anomalous variables to perform root cause analysis and promptly address system issues. For example, Moon’s explainer highlights variables 12 and 15 as anomalous, we can investigate potential disk bottlenecks by examining which processes are heavily utilizing disk I/O and take corresponding optimizations.

VI. RELATED WORK

Modality Conversion: Time series data can be transformed into time-frequency representations using various modal conversion methods, such as Short-Time Fourier Transform (STFT) [45], Wavelet Transform [37], and Hilbert-Huang Transform (HHT) [24]. Each method has distinct characteristics: STFT uses a fixed window scale, which cannot adjust based on frequency; WT is better suited for non-stationary signals; and HHT offers adaptability but with lower resolution.

Moreover, time series data can be encoded into images and feature extraction and classification can be performed using Convolutional Neural Networks (CNN). Common techniques include Gramian Angular Field (GAF) [47], Recurrence Plots (RP) [16], Markov Transition Field (MTF) [47], Difference Field [52], and Relative Position Matrix [7]. GAF converts time series data into cosine and sine Gramian matrices, preserving angular information and capturing dynamic features. RP generates two-dimensional matrices reflecting similarity structures, making it suitable for identifying repeating patterns and periodicity. MTF creates Markov transition matrices, generating images that reflect state transitions and capture dynamic evolution. DF highlights data trend changes by producing images through differential operations. RPM maps time series data to high-dimensional spaces to generate images reflecting relative positional relationships. Among these, MTF is particularly effective for time series data with strong contextual links, as it captures dynamic characteristics and evolutionary trends. However, current research on multivariate MTF techniques remains limited.

To address this gap, this paper proposes MV-MTF that transforms MTS with contextual links into two-dimensional images. MV-MTF preserves transition relationships between variables over time, effectively enhancing anomaly detection.

Deep Learning Anomaly Detection. Deep learning models [5], [35], [41], [42], [47] have demonstrated exceptional effectiveness in anomaly detection tasks for complex and large-scale datasets due to their ability to learn intricate data representations and features.

OmniAnomaly [39] utilizes a stochastic recurrent neural network combined with a planar normalizing flow to generate reconstruction probabilities. It integrates an improved Peak Over Threshold (POT) method for automated anomaly threshold selection, significantly enhancing performance. However, its training process is lengthy and resource-intensive. MAD-GAN [26] employs an LSTM-based GAN model to capture time-series distributions through its generator, incorporating both prediction error and discriminator loss into anomaly scoring. This method

boosts anomaly detection performance but can also be computationally demanding.

USAD [5] is more efficient than the above methods because it utilizes a simple autoencoder with dual decoders trained in an adversarial framework. This lightweight design significantly reduces training time while maintaining competitive performance. TranAD [42] incorporates focus score-based self-conditioning for robust multimodal feature extraction, adversarial training for stability, and Model-Agnostic Meta-Learning (MAML) for effective training on limited data.

CNNs have proven highly effective in extracting and learning features from data, enabling them to capture complex structures. Omni-Scale CNN [41] is particularly adept at handling multi-scale MTS data through its multi-scale feature extraction capabilities. However, it does not support multimodal data, limiting its applicability to more diverse datasets.

To address this limitation, we enhance Omni-Scale CNN by incorporating parameter sharing and a feature fusion module, creating a multimodal anomaly detection model. This enhanced model significantly improves its ability to process and analyze multimodal data, further advancing its effectiveness in anomaly detection.

Explainable Anomaly Detection: Explainable Anomaly Detection (XAD) extracts insights from anomaly detection models, emphasizing data relationships to help users understand the causes of anomalies, thereby enhancing interpretability and usability. In time-series data, XAD methods can be classified into sample-based and model-based methods.

Sample-based methods explain anomalies by comparing anomalous and normal objects, focusing on local neighborhoods, counterexamples, or contextual anomalies. For example, reconstruction-based methods such as OmniAnomaly [39] and TranAD [42] identify anomalous variables by comparing reconstructed values with original ones.

Model-based methods explain the internal workings or predictions of anomaly detection models. They can be further divided into glass-box and closed-box methods. Glass-box methods are inherently interpretable, while closed-box methods use post-hoc techniques, often model-agnostic, to explain predictions [4], [36]. For example, closed-box methods such as SHAP values [31], [33] provide insights into feature importance within the model. We integrate sample-based and model-based methods by leveraging SHAP values to enhance interpretability and using normal samples to define the range of anomalous variables.

VII. CONCLUSION

This paper introduces MOON, an efficient and effective framework for multivariate time series anomaly detection. MOON utilizes the MV-MTF technology to provide more detailed multi-modal information via mode conversion, while introduces Multimodal-CNN to effectively learns and integrates information from different modalities. Additionally, MOON provides user-friendly anomaly interpretability by using SHAP values to rank variables in ascending order of their impact on anomalies and select the top-ranked ones. Extensive experiments evaluate the performance of Moon and validate the effectiveness of

each component/technique in MOON. compared with existing state-of-the-arts methods, MOON achieves high efficient and accurate anomaly detection and provides the detail anomaly analysis report for good interpretability. In the future, it is of interest to extend our anomaly detection model to handle other downstream tasks.

REFERENCES

- [1] Shap, 2024. Accessed: Aug. 2, 2024. [Online]. Available: <https://shap.readthedocs.io/en/latest/>
- [2] A. Abdulaal, Z. Liu, and T. Lancewicki, "Practical approach to asynchronous multivariate time series anomaly detection and localization," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2485–2494.
- [3] M. Abououf, S. Singh, R. Mizouni, and H. Otrouk, "Explainable AI for event and anomaly detection and classification in healthcare monitoring systems," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 3446–3457, Jan. 2024.
- [4] R. Assaf and A. Schumann, "Explainable deep neural networks for multivariate time series predictions," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 6488–6490.
- [5] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, R. Gupta, Y. Liu, J. Tang, and B. A. Prakash, Eds., 2020, pp. 3395–3404.
- [6] A. Bhatta, D. Mery, H. Wu, J. Annan, M. C. King, and K. W. Bowyer, "Our deep CNN face matchers have developed achromatopsia," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2024, pp. 142–152.
- [7] W. Chen and K. Shi, "A deep learning framework for time series classification using relative position matrix and convolutional neural network," *Neurocomputing*, vol. 359, pp. 384–394, 2019.
- [8] X. Chen et al., "DAEMON: Unsupervised anomaly detection and interpretation for multivariate time series," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 2225–2230.
- [9] E. Çökaj, H. S. Gustad, A. Leone, P. T. Moe, and L. Moldestad, "Supervised time series classification for anomaly detection in subsea engineering," 2024, *arXiv:2403.08013*.
- [10] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Min. Knowl. Discov.*, vol. 34, no. 5, pp. 1454–1495, 2020.
- [11] A. Dempster, D. F. Schmidt, and G. I. Webb, "Hydra: Competing convolutional kernels for fast and accurate time series classification," *Data Min. Knowl. Discov.*, vol. 37, no. 5, pp. 1779–180, 2023.
- [12] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 4027–4035.
- [13] N. Ding, H. Ma, H. Gao, Y. Ma, and G. Tan, "Real-time anomaly detection based on long short-term memory and gaussian mixture model," *Comput. Electr. Eng.*, vol. 79, 2019, Art. no. 106458.
- [14] G. B. Gaggero, A. Armellini, G. Portomauro, and M. Marchese, "Industrial control system-anomaly detection dataset (ICS-ADD) for cyber-physical security monitoring in smart industry environments," *IEEE Access*, 12, pp. 64140–64149, 2024.
- [15] M. Gupta, J. Gao, C. C. Aggarwal, and J. Han, "Outlier detection for temporal data: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 9, pp. 2250–2267, Sep. 2014.
- [16] N. Hatami, Y. Gavet, and J. Debayle, "Classification of time-series images using deep convolutional neural networks," in *Proc. 10th Int. Conf. Mach. Vis.*, A. Verikas, P. Radeva, D. P. Nikolaev, and J. Zhou, Eds., 2017, vol. 10696, Art. no. 106960.
- [17] M. Huang, C. Nie, and W. Zhong, "A visualization method for data domain changes in CNN networks and the optimization method for selecting thresholds in classification tasks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2024, pp. 986–994.
- [18] A. Huet, J. M. Navarro, and D. Rossi, "Local evaluation of time series anomaly detection algorithms," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, A. Zhang and H. Rangwala, Eds., 2022, pp. 635–645.
- [19] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.

- [20] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, "Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [21] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
- [22] S. Kim, K. Choi, H. Choi, B. Lee, and S. Yoon, "Towards a rigorous evaluation of time-series anomaly detection," in *Proc. 36th AAAI Conf. Artif. Intell.*, 2022, pp. 7194–7201.
- [23] D. P. Kingma and J. B. Adam, "A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Representations*, vol. 5, no. 6, 2015.
- [24] V. G. Kurbatsky, D. N. Sidorov, V. A. Spiryayev, and N. V. Tomin, "Forecasting nonstationary time series based on Hilbert-Huang transform and machine learning," *Autom. Remote. Control.*, vol. 75, no. 5, pp. 922–934, 2014.
- [25] H. Lee and J. Lee, "Convolutional model with a time series feature based on RSSI analysis with the markov transition field for enhancement of location recognition," *Sensors*, vol. 23, no. 7, 2023, Art. no. 3453.
- [26] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Artif. Neural Netw. Mach. Learn.*, in Lecture Notes in Computer Science, 2019, vol. 11730, pp. 703–716.
- [27] L. Li, X. Lin, B. Ran, and B. Du, "Tensor decomposition of transportation temporal and spatial Big Data: A brief review," *Fundam. Res.*, early access, Sep. 25, 2024, doi: [10.1016/j.fmre.2024.03.032](https://doi.org/10.1016/j.fmre.2024.03.032).
- [28] C. Lin, B. Du, L. Sun, and L. Li, "Hierarchical context representation and self-adaptive thresholding for multivariate anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3139–3150, Jul. 2024.
- [29] H. Liu and L. Li, "Anomaly detection of high-frequency sensing data in transportation infrastructure monitoring system based on fine-tuned model," *IEEE Sensors J.*, vol. 23, no. 8, pp. 8630–8638, Apr. 2023.
- [30] H. Liu, W. Luo, L. Han, P. Gao, W. Yang, and G. Han, "Anomaly detection via graph attention networks-augmented mask autoregressive flow for multivariate time series," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19368–19379, Jun. 2024.
- [31] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
- [32] A. P. Mathur and N. O. Tippenhauer, "SWAT: A water treatment testbed for research and training on ICS security," in *Proc. CySWater*, 2016, pp. 31–36.
- [33] F. Mujkanovic, V. Doskok, M. Schirneck, P. Schäfer, and T. Friedrich, "Timexplain - A framework for explaining the predictions of time series classifiers," 2020, *arXiv:2007.07606*.
- [34] G. Pang, J. Li, A. V. D. Hengel, L. Cao, and T. G. Dietterich, "AN-DEA: Anomaly and novelty detection, explanation, and accommodation," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 4892–4893.
- [35] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
- [36] M. T. Ribeiro, S. Singh, and C. Guestrin, "Anchors: High-precision model-agnostic explanations," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 30th Innov. Appl. Artif. Intell. Conf.; *Proc. 80th AAAI Symp. Educ. Adv. Artif. Intell.*, 2018, pp. 1527–1535.
- [37] L. Sasal, T. Chakraborty, and A. Hadid, "W-transformers: A wavelet-based transformer framework for univariate time series forecasting," in *Proc. 21st IEEE Int. Conf. Mach. Learn. Appl.*, M. A. Wani, M. M. Kantardzic, V. Palade, D. Neagu, L. Yang, and K. Y. Chan, Eds., 2022, pp. 671–676.
- [38] A. Song, E. Seo, and H. Kim, "Anomaly VAE-transformer: A deep learning approach for anomaly detection in decentralized finance," *IEEE Access*, vol. 11, pp. 98115–98131, 2023.
- [39] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.
- [40] C. W. Tan, A. Dempster, C. Bergmeir, and G. I. Webb, "Multirocket: Multiple pooling operators and transformations for fast and effective time series classification," *Data Min. Knowl. Discov.*, vol. 36, no. 5, pp. 1623–1646, 2022.
- [41] W. Tang, G. Long, L. Liu, T. Zhou, M. Blumenstein, and J. Jiang, "OMNI-scale CNNs: A simple and effective kernel size configuration for time series classification," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–17.
- [42] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1201–1214, 2022.
- [43] H. Wang, Z. Luo, J. W. L. Yip, C. Ye, and M. Zhang, "ECGGAN: A framework for effective and interpretable electrocardiogram anomaly detection," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 5071–5081.
- [44] J. Wang et al., "Real-time workload pattern analysis for large-scale cloud databases," *Proc. VLDB Endow.*, vol. 16, no. 12, pp. 3689–3701, 2023.
- [45] T. Wang and F. Kirchner, "Grasp stability prediction with time series data based on STFT and LSTM," in *Proc. Int. Conf. Adv. Robot. Mechatron.*, 2023, pp. 587–593.
- [46] Z. Wang and T. Oates, "Encoding time series as images for visual inspection and classification using tiled convolutional neural networks," in *Proc. Workshops 29th AAAI Conf. Artif. Intell.*, 2015, pp. 91–96.
- [47] Z. Wang and T. Oates, "Imaging time-series to improve classification and imputation," in *Proc. 24th Int. Conf. Artif. Intell.*, Q. Yang and M. J. Wooldridge, Eds., 2015, pp. 3939–3945.
- [48] R. Wu and E. J. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2421–2429, Mar. 2021.
- [49] X. Wu, "CATCH: channel-aware multivariate time series anomaly detection via frequency patching," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 56894–56922.
- [50] Y. Yao, H. Jie, L. Chen, T. Li, Y. Gao, and S. Wen, "TSec: An efficient and effective framework for time series classification," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 1394–1406.
- [51] M. Zhang, B. Xu, and J. Gong, "An anomaly detection model based on one-class SVM to detect network intrusions," in *Proc. 11th Int. Conf. Mobile Ad-hoc Sensor Netw.*, 2015, pp. 102–107.
- [52] Y. Zhang and X. Chen, "Motif difference field: A simple and effective image representation of time series for classification," 2020, *arXiv:2001.07582*.
- [53] Y. Zhao et al., "A comparative study on unsupervised anomaly detection for time series: Experiments and analysis," 2022, *arXiv:2209.04635*.



Yuanyuan Yao received the master's degree in computer science from the University of Chinese Academy of Sciences, China, in 2018. She is currently working toward the PhD degree in the College of Computer Science, Zhejiang University, China. Her research interests include time series management, compression, and analytics.



Yuhuan Shi received the BS degree in computer science from the Nanjing University of Aeronautics and Astronautics, China, in 2024. She is currently working toward the master's degree in the College of Software, Zhejiang University, China. Her research interests include anomaly detection in time series data and distributed databases.



Lu Chen received the PhD degree in computer science from Zhejiang University, Hangzhou, China, in 2016. She was an assistant professor in Aalborg University for a 2-year period from 2017 to 2019, and she was an associated professor in Aalborg University for a 1-year period from 2019 to 2020. She is currently a ZJU Plan 100 professor in the College of Computer Science, Zhejiang University. Her research interests include indexing and querying metric spaces.



Ziquan Fang (Member, IEEE) received the PhD degree in computer science from Zhejiang University, Hangzhou, in 2023. He is currently a researcher with Zhejiang University's "Hundred Talents Program". His research interests include big trajectory data management, spatio-temporal data mining, and distributed streaming data processing.



Yunjun Gao (Senior Member, IEEE) received the PhD degree in computer science from Zhejiang University, China, in 2008. He is currently a professor in the College of Computer Science, Zhejiang University. His research interests include spatial and spatio-temporal databases, metric and incomplete/uncertain data management, graph databases, spatio-textual data processing, and database usability. He is a member of the ACM and a senior member of IEEE.



Leong Hou U (Member, IEEE) received the PhD degree in computer science from the University of Hong Kong, Hong Kong, in 2010. He is currently an assistant professor with the University of Macau. His research interest include spatial and spatio-temporal databases, advanced query processing, web data management, information retrieval, data mining, and optimization problems.



Yushuai Li (Senior Member, IEEE) received the PhD degree in control theory and control engineering from the Northeastern University, Shenyang, China, in 2019. He is currently an assistant professor with the Department of Computer Science, Aalborg University. He is also an associate editor for *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Automation Science and Engineering*, *Journal of Modern Power Systems and Clean Energy*, and *IEEE Systems, Man, and Cybernetics Magazine*. His main research interests include machine learning, digital twin, and integrated energy.



Tianyi Li (Member, IEEE) received the PhD degree from Aalborg University, Denmark, in 2022. She currently an assistant professor with the Department of Computer Science, Aalborg University. Her main research interests include data management and analytics, machine learning, and digital twin. She is also an associate editor for *IEEE Transactions on Intelligent Vehicles* and *IEEE Network*. She is a member of IEEE.